Interaction Scaling: A Third Test-Time Compute Axis Grounded in Environment Feedback

Bojie Li
Pine AI
boj@19pine.ai

Abstract

Test-time compute is dominated today by two scaling axes: *reasoning* (longer chains of thought) and *sampling* (best-of-N). Both operate inside the model's own token space and are bounded by the information already encoded in its weights. A third axis — iterating against an external environment that returns grounded feedback (execution traces, rendered artifacts, search verdicts) — is well established empirically (Reflexion, ReAct, CRITIC, RLEF, thinking-vs-doing) but lacks a unified theoretical account of *why* and *when* it should outperform reasoning- or sampling-only scaling. We supply that account: an information-theoretic formalization that derives a data-processing-inequality ceiling on ungrounded review and shows that grounded feedback channels break it, organized through a Type 0/1/2/3 feedback taxonomy with derived ordering predictions. We instantiate the framework with a budget-aware proposer–reviewer harness that wraps a frontier model without fine-tuning. Across six modalities, the harness lifts quality on every modality, with the largest and most reliable gain on the execution-grounded channel — code (+33.3 pp pass-rate, to a strict 100% across seeds) — as the taxonomy predicts, isolated further by an in-modality control that holds the task suite fixed and varies only the feedback type. A central methodological finding is that *evaluating* the visual modalities is itself a grounding problem: a vision–language-model judge reading a rendered screenshot is unreliable for the geometric defects that dominate generated figures and slides (it rates 14 of 15 dense academic-figure renders "perfect" when only 3 are actually clean, because content that overflows the canvas is cropped off-frame before the judge ever sees it). We therefore evaluate fixed-canvas visual artifacts with a *deterministic* DOM-geometry check — measuring every element's true bounding box to compute overlaps, clipping, and overflow exactly — and show that grounding the reviewer in the same signal drives a 78 % reduction in real layout defects on a new academic-figure modality (1.20 → 0.27 defects/figure, sign-test p=0.008, zero regressions) and a 91 % reduction on slides — gains the screenshot-judge metric could not even measure. At a 20K-token budget on hard code tasks, reasoning-only saturates at 73.3% (86.7% under a more generous thinking-heavy partition) and best-of-N at 86.7%, while iteration with environmental feedback — whether single-agent (L), sample-and-select (IAD), or proposer–reviewer (H) — reaches $\geq$ 97.8%. Among interaction strategies the proposer–reviewer harness is the most token-efficient (1,029 vs 1,431 L vs 1,416 IAD mean output tokens) and the only variant tested at strict 100% across multiple seeds (L hits 100% in 2 of 3 seeds; IAD was run for one seed only). The effect replicates across model families (Sonnet 4 +33.3 pp, Qwen3-235B +26.7 pp, GPT-5 +13.3 pp on a higher single-shot baseline) and to held-out tasks (90.6 → 100%, 3/3 failures recovered, zero regressions); the optimal token allocation is propose-heavy and the allocation

simplex shows an 86.6 pp spread. We additionally show a recipe for *internalizing* the harness's interaction-quality behavior into an 8B student via SFT on judge-filtered teacher trajectories: 0.50× teacher mean@1 capability on a 44-task out-of-distribution suite (rising to 0.70× at pass@2), and 44 %/56 % pass@1/pass@3 on an 18-task hard held-out suite, at ∼10× lower deployment cost than running the harness over a frontier model. Interaction scaling is real, distinct from reasoning and sampling scaling, predictable from the taxonomy, and operational both as scaffolding and (partially) as distilled behavior.

1 Introduction

Test-time compute has become the dominant lever for improving large-model performance on artifact-producing tasks — code generation, web pages, slide decks, video edits, research reports. The two scaling axes that the literature has formalized so far both operate *inside the model's own token space*. *Reasoning scaling* (chain-of-thought, tree search, extended thinking; 6, 24, 26, 30, 31) generates more thinking tokens before committing to an answer. *Sampling scaling* (best-of-N, self-consistency; 2, 29) generates multiple independent attempts and picks one. In both regimes, every additional token is a function of the same fixed weights and the same fixed task description: more compute does not bring more information into the system.

This is consequential for tasks where the failure mode is missing information, not missing computation. A program may type-check and read correctly but fail a test the model has no way to enumerate from the spec alone. A web page may have plausible HTML but render with the hero element clipped below the fold. A factual claim may sound right but be contradicted by a source the model never read. For these failures, longer thinking cannot help, because there is no signal inside the weights that distinguishes the correct from the incorrect output — the distinguishing signal lives outside the model. A long literature corroborates this intuition: LLMs cannot reliably self-correct without external feedback [12, 13], multi-agent debate gains collapse under matched-compute single-agent budgets [27], and tool-grounded verification dramatically outperforms self-evaluation [10].

Thesis. Test-time compute has a third scaling axis: *interaction scaling*, in which the model iteratively interacts with an external environment that returns grounded feedback (program output, rendered pixels, search results, lint warnings). The empirical observation is not new — Reflexion, ReAct, CRITIC, RLEF, and recent thinking-vs-doing work all instantiate it on specific modalities. What we contribute is the *formalization*: an information-theoretic account showing that each grounded interaction cycle injects information that was *physically outside* the model's weights at generation time, and that this breaks the information-theoretic ceiling bounding reasoning- and sampling-only scaling — not as a heuristic claim but as a direct consequence of the data processing inequality applied to the appropriate graphical model (Section 3). The formalization yields a feedback taxonomy with derived ordering predictions, which we then validate empirically across six modalities and three model families.

Four contributions. We make this claim precise and demonstrate it empirically. *First*, we formalize a *grounded feedback framework* (Section 3): a Type 0/1/2/3 taxonomy of feedback channels and an information-theoretic account that predicts *which* feedback channels can break the DPI ceiling and *which* cannot, with a derived ordering on the magnitude of the interaction-scaling lift. *Second*, we instantiate the framework as a *budget-aware proposer–reviewer harness* (Section 4) wrapped around a frozen frontier model — pure scaffolding, no fine-tuning, no learned controller —

and evaluate it on six modalities spanning all four sub-types of Type 3 feedback (Section 5). The harness lifts quality on every modality; the lift is largest exactly where the framework predicts (execution-grounded, +33.3 pp on code); among four strategies tested (reasoning-only, sampling, single-agent loop, harness) on hard code at the 20K-token budget, it is the only one to reach strict 100% pass-rate *with zero seed variance and the lowest mean token cost* (1,029 vs. 1,431 for the single-agent loop and 1,416 for an oracle-verifier sample-and-select baseline); and it replicates across three model families (Anthropic, Alibaba, OpenAI) and to a held-out task set. *Third*, we establish that for visual modalities the *evaluation* is itself a grounding problem and resolve it (Section 6): screenshot-reading VLM judges — holistic scores or even binary per-requirement rubrics — systematically miss the geometric defects (text-on-text overlap, clipped labels, canvas overflow) that dominate generated figures and slides, because the failures are literally cropped out of the captured image. We replace the judge with a *deterministic DOM-geometry* check on fixed-canvas artifacts and introduce an academic-figure modality on which grounded geometric feedback cuts real layout defects by 78 % (zero regressions, p=0.008); the same objective lens shows the slide harness was removing 91 % of defects all along, invisibly to its own VLM score. *Fourth*, we show a recipe for *internalizing* the harness's interaction-quality behavior into an 8B student (Section 7) by SFT on judge-filtered teacher trajectories, recovering 0.50× teacher mean@1 capability on a 44-task out-of-distribution suite (44 %/56 % pass@1/pass@3 on an 18-task hard held-out suite) at ∼10× lower deployment cost.

Roadmap. Section 2 surveys related work on test-time scaling, self-correction, and tool-grounded verification. Section 3 develops the feedback taxonomy and the DPI argument. Section 4 specifies the harness. Section 5 reports the main empirical results (six modalities, four-curve scaling, allocation, cross-model, held-out). Section 6 re-examines the visual modalities with a deterministic geometric metric, introduces the academic-figure modality, and reports the objective defect-reduction results. Section 7 reports the 8B distillation positive results. Section 8 discusses limits and implications; Section 9 concludes.

2 Related Work

Test-time scaling. A growing body of work shows that allocating extra compute at inference — via chain-of-thought [30], self-consistency [29], tree search [31], iterative refinement [16, 23], or compute-optimal allocation [1, 24] — improves task performance well past single-shot ceilings. The most recent reasoning-trained systems (DeepSeek-R1, Kimi K1.5; 6, 26) demonstrate that test-time-scaled behavior can be elicited via large-scale RL on a strong base. These approaches operate primarily on the reasoning and sampling axes. Our work argues that a third axis — interaction scaling, in which extra compute buys cycles of grounded environmental feedback rather than tokens inside the model — has fundamentally different scaling properties.

Self-correction limits and grounded verification. Huang et al. [12] showed that LLMs cannot reliably self-correct reasoning errors without external feedback; accuracy often *decreases* after self-review. Kamoi et al. [13] confirmed this across a broad range of settings: self-correction works only when (i) external feedback is available or (ii) the model has been fine-tuned for correction. Gou et al. [10] provided a controlled comparison: tool-grounded verification (CRITIC) yields substantial gains, but removing the tool and relying on self-evaluation eliminates most improvement. Multi-agent debate [7] appears to gain ground but Tran and Kiela [27] show that under matched-compute budgets a single agent matches debate on multi-hop reasoning — the gains attributed to cross-LLM critique come from the extra compute, not the cross-LLM signal. These results converge on the

same conclusion that our framework formalizes: feedback only breaks the information ceiling when it is grounded outside the model's weights.

Thinking vs. doing. Shen et al. [22] introduced an explicit distinction between scaling per-step reasoning (*thinking*) and scaling environment interaction (*doing*) for web agents, showing that under a fixed token budget, acting longer yields larger gains than thinking longer. Our work refines this with a third component — *reviewing*, a structured evaluation step that converts raw execution output into actionable improvement signals — and generalizes the empirical claim across six modalities (code, web, slides, video, research, animations).

Agentic LLMs and grounded refinement. Tool-augmented LLMs [19, 21] and agentic frameworks that interleave reasoning and acting [28, 32] are the dominant deployment paradigm for tasks that require external feedback. Execution-grounded refinement has been particularly impactful for code (RLEF; 9) and for web/visual generation (WebGen-Agent; 15). Our contribution relative to these works is the formal characterization of *why* they work — the DPI argument applied to the feedback channel — and a systematic study of the architectural and budget choices that determine when they help.

Sampling-based inference scaling and consistency-vs-diversity. Best-of-n and pass@k inference [2, 4] rely on output diversity. A long line of work documents that fine-tuning, and especially RLHF [3, 25], can collapse this diversity along multiple axes [14, 18]. We confirm and extend this story in Section 7: a rejection-sampling fine-tune that improves every per-turn consistency metric we measure regresses pass@3 by 17 pp.

Distillation of tool use and refinement behavior. A separate line distills test-time-scaled behavior into smaller students [8, 11, 17]. DeepSeek-R1's distilled checkpoints [6] and Kimi K1.5 [26] show that high-quality reasoning traces *can* distill at scale. Most of this work is text-only; we add the multimodal interaction-quality dimension and a recipe specific to the small-student regime (Section 7).

3 The Grounded Feedback Framework

We propose an information-theoretic framework that predicts when adding a review step to an LLM agent helps and when it does not. The key distinction is whether the review step's output is a function only of the model's existing internal state, or of new information that originates outside the model's weights.

3.1 Feedback taxonomy

We distinguish four feedback types by their information source.

- **Type 0 — Self-review (no grounded information).** The same model re-reads its own output. The reviewer operates on the same weights and the same context as the proposer. Empirically, this often *decreases* accuracy [12, 13]. Analogous to re-reading your own essay without showing it to anyone.
- **Type 1 — LLM cross-review (no grounded information).** A different LLM (or the same LLM with a different prompt) critiques the artifact. Although the reviewer brings different priors, it operates only on the textual representation of the artifact, with no external verification. Tran

and Kiela [27] show that under matched compute this provides no advantage over a single-agent baseline. Analogous to re-reading your essay wearing a different hat.

- **Type 2 — Static tool feedback (grounded, pre-execution).** External tools analyze the artifact without executing it: linters, type checkers, structural validators. The information is grounded in formal language specifications and rule sets but is restricted to properties verifiable without runtime behavior.
- **Type 3 — Dynamic environment feedback (grounded, post-execution).** The artifact is executed, rendered, or verified against external sources, and the results are fed back to the reviewer. Type 3 has four sub-modalities corresponding to the natural feedback channel for each artifact type:
  - Type 3a: execution feedback (test pass/fail, traceback, runtime behavior).
  - Type 3b: visual rendering feedback (screenshot of a rendered artifact analyzed by a VLM).
  - Type 3c: temporal feedback (keyframe sequence from video/animation, analyzed for temporal coherence).
  - Type 3d: factual verification feedback (search-engine or knowledge-base lookups verifying or contradicting specific claims).

3.2 Information-theoretic formalization

We make the difference between Type 0/1 and Type 2/3 feedback precise with an explicit graphical model.

Random variables. Let T denote the task specification, A the generated artifact, E the environment's response (execution result, rendered pixels, search verdict, lint output; null for Type 0/1), F the reviewer's feedback delivered to the proposer for revision, and R the final outcome / reward (does the revised artifact satisfy T?). The quantity of interest is $I(R; F \mid A)$ — how much information about whether the revision will succeed is recoverable from F beyond what is already in A. If this is zero, the reviewer adds no decision-relevant signal and revisions are bounded above by what the proposer could have achieved without review.

Type 0/1 (no grounding). The reviewer reads only A and T; there is no external variable. The graphical model is the Markov chain $T \to A \to F$, with R a function of A (and unobserved stochastic execution). The defining conditional-independence assumption is

$$F \perp R \mid A,$$

i.e. conditioning on the artifact, the feedback adds no information about the eventual outcome. LLM sampling makes F random, but the noise is uncorrelated with task-relevant facts about whether A works. By the data processing inequality [5],

$$I(R; F) \leq I(R; A).$$

The reviewer cannot manufacture task-relevant information that was not already encoded in A. Self-review and pure LLM cross-review reshuffle the same bits.

Type 2/3 (grounded). The graphical model gains the external variable E, giving $T \to A \to E$ and $F = f(T, A, E)$. Crucially, E is generally *not* a deterministic function of (T, A): the environment carries information about A's correctness that is *not deducible from A's surface form alone.* A program that "looks correct" may fail tests; HTML that "parses correctly" may render with the hero clipped; a factual claim that "sounds right" may be contradicted by a source the model has not read. The conditional-independence assumption $F \perp R \mid A$ no longer holds, and

$$I(R; F) \;>\; I(R; A) \quad \text{is achievable.}$$

The environment breaks the Markov chain by injecting a genuinely new, reliable information source into F.

What this argument proves vs. predicts. The DPI bound is a *qualitative* statement: Type 0/1 feedback cannot exceed the single-shot information ceiling $I(R; A)$, while Type 3 feedback can. A subtlety made visible by the in-modality controls in Section 5.5 is worth stating up front: the DPI bound is on F's information about R *beyond what is already in* A; it is *not* a bound on what a single proposer forward-pass extracts from A. A different reviewer prompt may surface information from A that the proposer's prompt did not — a different projection of the same artifact, bounded above by $I(R; A)$. This is the fraction of the lift that an ungrounded Type 1 reviewer can deliver in practice; it is real but bounded. The qualitatively new axis Type 3 unlocks is information *not in A at all* (the artifact's actual runtime behavior), which only an external E can supply.

The bound also does *not* predict the magnitude of the gain inside Type 3, nor the within-tier ordering across Type 3a/b/c/d. Whether execution feedback beats visual feedback by a factor of two or ten is determined by how much of the environment's information F encodes (encoder fidelity), how localizable the encoded signal is (actionability), and how reliably the proposer can translate F into a corrective edit (action translation) — none of which the framework constrains. Those are empirical questions, addressed in Sections 5.4 and 5.7. The framework's role is to identify the load-bearing axis (presence vs. absence of an external E) and to rule out the regime in which longer self-review and longer reasoning chains can substitute for it.

3.3 Predictions

Three predictions follow from this framework, and the empirical sections test each.

1. *Type 0/1 reviewers cannot exceed $I(R; A)$ and should fall short of grounded Type 3 channels at matched compute.* Prior work [12, 27] already shows that ungrounded review is bounded; our refinement (Section 5.5) is that a Type 1 reviewer can still lift quality *up to* $I(R; A)$ by surfacing information from A that the proposer's prompt did not, and is reliably surpassed by Type 3a (which adds genuinely new information about A's runtime behavior).

2. *Type 3 reviewers should lift quality, with a tier-level gap between execution-grounded and non-execution channels.* Execution feedback (Type 3a) is binary, machine-readable, and localizable; visual feedback (Type 3b/3c) and factual verification (Type 3d) are softer (a holistic critique or a per-claim verdict does not localize to a specific edit). The framework predicts the tier-level grouping Type 3a $\gg$ {Type 3b, 3c, 3d}; it does *not* predict the within-tier ranking, which is sensitive to task headroom, feedback noise, and action-translation losses (Section 5.4). We test the tier-level prediction directly in Section 5.4.

3. *Reasoning-only scaling at matched compute should fall short of the interaction-scaling ceiling.* Longer chains of thought operate within the DPI bound; for failures where A alone does not determine correctness (off-by-one boundary conditions revealed by a test, layout overflows revealed

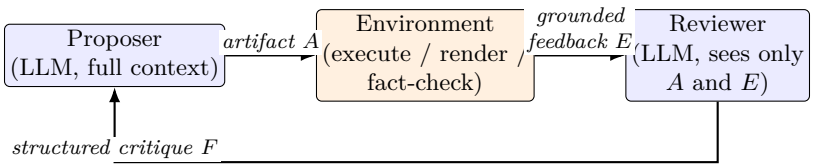


Figure 1: The proposer–reviewer harness. Blue boxes are LLM calls (same model, different contexts); orange is the environment. The reviewer's context is reset between iterations to contain only the latest artifact and the grounded feedback channel — preventing context overload and concentrating the LLM's analysis on the localizable signal in E.

by a render), reasoning cannot manufacture the missing signal. We test this in Sections 5.6 and 5.7.

4 Method: The Proposer–Reviewer Harness

4.1 Architecture

The proposer–reviewer harness instantiates the framework as an inference-time scaffold (Figure 1). A frozen model serves as both proposer and reviewer; what distinguishes the two roles is the context they operate on, not the weights.

- The **proposer** generates an initial artifact A from the task spec T.
- The **environment** executes / renders / fact-checks A, producing the grounded feedback variable E (Type 3a/b/c/d depending on the modality).
- The **reviewer** consumes only the latest artifact A and the grounded feedback E — *not the full conversation history* — and emits a structured critique F identifying defects.
- The proposer revises in a fresh context informed by F and the prior artifact.

The architectural choice that distinguishes the harness from a single-agent loop is the reviewer's context isolation. A single-agent loop accumulates execution output and revision turns inside one ever-growing context; the harness resets the reviewer's input to the minimum information needed to localize defects. Section 5.8 shows this choice ties the single-agent loop on pass rate at saturation while spending fewer tokens (1,029 vs. 1,431 mean) and exhibiting zero seed variance.

4.2 Budget definition and allocation

Let B denote the total token budget per task (summed across all LLM calls). The harness splits B into three slots:

$$b_1 \text{ (propose)} \;+\; b_2 \text{ (execute)} \;+\; b_3 \text{ (review)} \;\leq\; B.$$

For code, b_2 is effectively zero (`pytest` consumes no LLM tokens); for the visual modalities, b_2 is the VLM call that produces the rendered-screenshot critique. Section 5.11 shows that under a tight per-task budget the dominant axis is b_1: the proposer must be given enough per-call room to emit a complete artifact, and starving it (small b_1) collapses pass-rate regardless of how much budget the reviewer gets.

4.3 Cross-modal instantiation

The same architecture instantiates differently per modality, distinguished by the grounded feedback channel.

Table 1: Modality–specific instantiations of the harness. The feedback channel determines which Type 3 sub-modality the reviewer consumes; the proposer always sees text or images.

Modality	N tasks	Feedback channel	Max iters
Code	15	Type 3a: `pytest` pass/fail + traceback	5
Video editing	15	Type 3a+3c: script exec + keyframe VLM critique	3
Deep research	15	Type 3d: per-claim factual verification via search	2
Web pages	15	Type 3b: headless-browser screenshot + VLM critique	3
Animations	15	Type 3c: rendered keyframe sequence + temporal VLM	3
Slide decks	20	Type 3b: rendered slide images + per-slide VLM critique	3

5 Interaction Scaling with External Scaffolding

This section reports the main empirical results. Throughout, the harness is pure scaffolding — no fine-tuning, no retrieval-augmented context, no learned controller — so any gain over single-shot is attributable to interaction scaling alone.

5.1 Setup

Model. Claude Sonnet 4 (`claude-sonnet-4-20250514`), extended thinking enabled, temperature 0. The same model serves as proposer and reviewer; only the context differs. The harness uses no fine-tuning, retrieval, or learned controller. A cross-family replication on Qwen3-235B (Section 5.9) confirms the effect is architectural, not Claude-specific.

Conditions. *Single-shot*: one proposer call, output scored. *Reviewed*: full proposer→execute/render→review→revise loop until the reviewer accepts the artifact or the iteration cap is hit. We replicate each modality three independent times (`onpolicy_run{1,2,3}`) with fresh harness state; non-determinism comes from the API on long reasoning traces, since temperature is fixed at 0. Code is scored binary (canonical `pytest` pass/fail); the other modalities use a fixed-rubric VLM/LLM judge returning a continuous score in $[0, 1]$.

5.2 Six-modality headline results

Table 2 reports the mean and standard deviation across the three independent on-policy runs per modality. The harness lifts quality on *every* modality, and the lift is largest exactly where the framework predicts: the execution-grounded code channel clears $p < 0.05$ on a one-sided paired sign test and delivers the dominant lift (+33.3 pp to a strict 100%). The remaining modalities register smaller lifts in this overview, but it understates two of them: Section 6 measures the visual modalities with a deterministic instrument and shows the harness removing real layout defects by 78–91 % on figures and slides, and sharpening a significant rubric lift on web (+0.063, p=0.003) and animations (+0.192, p=0.008). Video editing, once scored by a native full-video rubric rather than the noisy holistic judge, is already strong single-shot and saturates (Section 5.3).

5.3 Capability emergence: code as a case study

The code result is the cleanest evidence that interaction scaling is a distinct axis of test-time compute rather than a quality knob, because the feedback channel is binary and the score is objective. Single-shot pass-rate is 66.7% (10/15); the harness lifts it to a strict 100% in all three runs, recovering every one of the five tasks that the first-shot draft gets wrong, with zero regressions. This is not a quality bump applied to working code: the single-shot draft *fails its tests*, and the interaction loop is what converts "plausible but wrong" into "passes the canonical suite."

Table 2: Single-shot vs. reviewed quality across six modalities, Claude Sonnet 4. Means and standard deviations are computed across three independent on-policy runs (per-run means, then aggregated). Code is binary pass-rate; other modalities are continuous in $[0, 1]$. p-values: one-sided paired sign test on per-task means. **Bold** marks rows where the lift clears $p < 0.05$. Code is scored by execution (`pytest`); video by a native full-video rubric (Gemini 3.1 Pro judging the whole rendered clip); the four visual/factual rows use a holistic screenshot judge and serve here as a cross-modal overview, with the precise web and animation figures given by the binary rubric in Table 12 and the academic-figure and slide results by deterministic geometry in Table 11.

Modality	N	Feedback type	Single-shot	Reviewed	Δ (sign-test p)
Code	15	3a	$66.7 \pm 6.7\%$	$\mathbf{100.0 \pm 0.0}\%$	$+\mathbf{33.3}$ pp (0.008)
Video editing	15	3a+3c	0.70 ± 0.03	0.74 ± 0.05	+0.04 (0.64)
Deep research	15	3d	0.63 ± 0.06	0.69 ± 0.06	+0.06 (0.090)
Web pages	15	3b	0.43 ± 0.03	0.56 ± 0.01	+0.13 (0.073)
Animations	15	3c	0.40 ± 0.03	0.55 ± 0.01	+0.15 (0.073)
Slide decks	20	3b	0.73 ± 0.01	0.79 ± 0.01	+0.06 (0.090)

Mechanism. Inspection of recovered traces shows a consistent pattern: turn 1 emits code that compiles but mishandles an edge case (off-by-one boundaries, semantic-version / natural-sort comparison, IP-address / CIDR arithmetic); turn 2 receives the `pytest` traceback — the failing assertion, the offending input, the expected vs. actual value — and rewrites the localized block; turn 3 confirms the fix against the suite. Reasoning alone cannot supply this signal — the model has no way to know its boundary condition is wrong until a concrete test exercises it. The information is physically outside the weights, and Section 5.5 confirms it is the *execution* of the tests, not merely an extra reviewing pass, that supplies it.

Video editing. Video editing is a second execution-grounded channel (the proposer writes an `ffmpeg`/`moviepy` script that is run, and the rendered clip is scored). We measure it with a native full-video rubric — Gemini 3.1 Pro judges each binary requirement against the whole rendered clip end-to-end — rather than the holistic keyframe judge, which we found to be a very noisy instrument for this modality. Under the rubric a frontier model's single-shot scripts are already largely correct (0.70, with 22/45 task-runs perfect), so the suite saturates and the reviewed lift is small and not significant (+0.04, p=0.64); the genuine recoveries are the few scripts that fail to produce a valid file. To restore measurement headroom we additionally construct a hardened suite of multi-step editing pipelines (frame-accurate trims, reverse-then-retime, speed ramps, grids, crossfade chains); it cuts the single-shot perfect rate from 49% to 11% with per-task difficulty spanning 0.44–0.90, confirming the original suite was saturated rather than the channel uninformative.

5.4 Feedback actionability validates the Type-3 taxonomy

Section 3.1 predicts a qualitative ordering between *tiers* of feedback grounding: a channel that *executes* the artifact and returns a binary, objective verdict (Type 3a) should lift more than channels that critique rendered output without exercising it (Type 3b/3c) and channels that verify factual claims without an executable consequence (Type 3d). The cleanest test holds the task suite fixed and varies only the feedback type: on the 15 hard code tasks, the Type 3a (execution) reviewer reaches a strict 100% where the Type 1/2 reviewers — which see cross-review and static-analysis output but never run the tests — stall at 13/15, and does so $\sim$2.5$\times$ more token-efficiently (Section 5.5). The cross-modality deltas in Table 2 corroborate this: the execution-grounded code channel delivers

the single largest lift (+33.3 pp to a strict 100%), and Section 6 shows that re-grounding the visual channels in a deterministic instrument moves them into the same high-actionability regime (78–91 % defect reduction).

The framework predicts the tier-level grouping, not a per-channel ranking. The DPI bound separates *grounded* from *ungrounded* feedback; the magnitude of the gain inside Type 3 is set by three factors the bound abstracts away — task headroom (research and slides start at 0.63/0.73 single-shot quality, web at 0.43), feedback fidelity (`pytest` return codes are cleaner than per-claim verdicts), and action-translation (a traceback localizes to an edit more directly than a holistic critique). These factors, not the taxonomy, govern the within-tier spread, and Section 6 shows that grounding the visual channel in deterministic geometry — which makes "the layout is broken" as localizable as a traceback — moves the visual modalities into the high-actionability regime.

The load-bearing claim is grounded vs. ungrounded feedback. The grouping the framework robustly predicts is *execution-grounded (and, after re-grounding, deterministic-geometric) feedback vs. ungrounded critique.* The in-modality control isolates it without any cross-modality confound: on one fixed code suite, executing the tests is what carries the reviewed ceiling to strict 100%, while an otherwise-identical reviewer that only reasons about the code does not (Section 5.5). *The feedback channel that runs the artifact and reports an objective verdict delivers the largest and most reliable interaction-scaling lift; channels that critique rendered output without exercising it help only once their signal is made as localizable as a traceback* (Section 6).

5.5 In-modality feedback-type controls (Type 1 / Type 2 / Type 3a)

Section 5.4 validates the tier-level prediction across modalities; the in-modality control fixes the task suite and varies only the feedback channel. On the same 15 hard code tasks we instantiate a Type 1 reviewer (LLM cross-review with no execution or static analysis), a Type 2 reviewer (LLM with `ruff` output appended; still no execution), and reuse the Type 3a baseline from Section 5.2. The Type 1/2 reviewers are forbidden from accessing test results and may emit `VERDICT: PASS` to early-stop.

Table 3: In-modality feedback-type controls on 15 hard code tasks (Sonnet 4 proposer and reviewer). Each row is a single on-policy seed; the "SS" column is the *per-seed* single-shot pass-rate (not held constant across rows) and "Reviewed" is the ceiling after that seed's harness run. The three SS values (10/15 and 11/15) are draws from the same single-shot distribution that gives Table 2's 3-run mean of $66.7 \pm 6.7\%$ (i.e. within the ± 1 SD band centered on 10/15). Because the SS column varies by row, the load-bearing comparison is the *reviewed* ceiling: Type 3a reaches 14/15, Type 1/2 reach 13/15 (+6.7 pp), *and* Type 3a is $\sim 2.5\times$ more token-efficient (1.53 vs. 2.07 iterations).

Type	Reviewer sees	SS	Reviewed	Iters
None (single-shot)	—	11/15	—	—
Type 1 (cross-review)	code only	10/15	13/15	2.07
Type 2 (+`ruff`)	code + static analysis	11/15	13/15	2.07
Type 3a (+exec)	code + test stderr/traceback	11/15	**14/15**	**1.53**

The pattern in Table 3 is informative on three axes the framework distinguishes.

Type 3a beats the static channels on capability and efficiency, and Type 1 commits the predicted ungrounded-channel failure mode. Type 3a's +6.7 pp reviewed-pass margin over Type 1/2 is one task on a 15-task suite (sign-test $p \approx 0.5$ in absolute terms), but the qualitative pattern matches the framework's prediction: Type 3a recovers `code_003` (a date-range overlap checker whose failing case — cross-timezone overlap and a zero-length range — the proposer cannot deduce without seeing the actual test traceback) where Type 1/2 do not, and is ∼2.5× more token-efficient (1.53 vs. 2.07 mean iterations) because the test verdict tells the proposer when to stop. On `code_003` the blind Type 1 reviewer falsely emitted `reviewer_verdict_pass` on broken code at iteration 3 (declaring the implementation "mathematically correct" and "will correctly handle the reported bug scenario" on code that in fact failed the cross-timezone test) — a false-positive verdict that is structurally impossible for Type 3a (the test verdict is the ground truth) and is the textbook ungrounded-channel failure mode the DPI argument predicts.

Type 1 is stronger than naive "no information" would predict, and Type 1 ≈ Type 2 by construction of the bug set. On a fixed 11/15 reference baseline the Type 1/2 reviewers each lift pass-rate to 13/15 (+13.3 pp), which is the fraction of the harness lift attributable to a different reviewer-prompt projection of A surfacing information the proposer's prompt missed (the refinement to the DPI argument noted in Section 3.2). On the Type 1 seed's own single-shot (10/15) the lift reads +20 pp, but this conflates two effects — a less-favorable single-shot draw and a real review lift — so we report the reviewed ceiling as the comparable quantity across rows. Type 1 and Type 2 do not separate because the 15 hard tasks are runtime-logic bugs (off-by-one boundaries, escape-handling, route-priority, byte-vs-char) that `ruff` cannot see; a benchmark deliberately seeded with bugs that `mypy` or `semgrep` catch would be the right place to test the strict Type 2 > Type 1 prediction, which this benchmark neither confirms nor refutes.

5.6 Reasoning-only at matched budget: is this just more compute?

A reviewer can object: "maybe the harness's gain is just from spending more tokens; did you let the single-shot model think longer?" We answer directly. On the same 15 hard code tasks, we run a reasoning-only baseline at matched budget: extended thinking enabled, `thinking_budget`=8000, no tools, no execution, no review. The single LLM call gets a generous over-budget for thinking; if reasoning-scaling alone explains the harness lift, this baseline should match it.

Table 4: Reasoning-only at matched budget vs. single-shot and the harness on the 15 hard code tasks. Single-seed numbers (the Phase-1 reference seed, also used in Table 3); the 3-run aggregate is reported in Table 2 ($66.7 \pm 6.7\%$ single-shot, $100.0 \pm 0.0\%$ reviewed). Reasoning closes two-thirds of the harness's gap over single-shot but falls 6.7 pp short of the harness's pass-rate even at 1.57× the harness's token budget.

Strategy	Pass-rate	Mean tokens	Δ vs. single-shot
Single-shot	73.3% (11/15)	1,406	—
Reasoning-only (matched budget)	86.7% (13/15)	4,137	+13.3 pp
Proposer–reviewer harness	**93.3% (14/15)**	**2,643**	**+20.0** pp

Reasoning-only closes two-thirds of the harness's gap over single-shot (+13.3 pp of the harness's +20.0 pp). The remaining 6.7 pp comes from a single task — `code_003`, a date-range overlap checker — where reasoning-only emits code that correctly handles naive vs. timezone-aware datetimes for the obvious cases but fails the zero-length range test (`AssertionError: Zero-length range should`

Table 5: **The headline scaling result (3-seed mean ± SD).** Pass-rate by strategy × budget on 15 hard code tasks, Sonnet 4. S, L, and H are averaged over three independent on-policy runs; R was run once (extended-thinking variation across seeds is small at temp 0) at the sweep's budget partition (a more generous partition lifts R to 86.7%; see footnote and Table 4). S's small dip from B=1K (80.0%) to B=5K (77.8%) is within the ±3.1 pp seed noise on a 3-seed × 15-task suite; pass@N is non-decreasing in N in expectation but not pointwise across finite-sample on-policy seeds. *Reasoning-only and best-of-N saturate well below 100%*; all three iteration-with-feedback strategies reach ≥ 97.8% at B=20K (L hit 100% on 2 of 3 seeds; H on all 3; H is the only variant at strict 100% across seeds). The architectural ordering across H, L is within seed noise on pass-rate; what separates them is token efficiency and reliability (Section 5.8).

Budget B	R (reasoning-only)	S (best-of-N)	L (single-agent loop)	H (proposer–reviewer)
1K	60.0%	80.0% ±0.0 pp	57.8% ±3.1 pp	62.2% ±3.1 pp
5K	73.3%	77.8% ±3.1 pp	**93.3%** ±0.0 **pp**	91.1% ±3.1 pp
20K	73.3%	86.7% ±0.0 pp	97.8% ±3.1 pp	**100.0%** ±0.0 **pp**
Ceiling	73.3%	86.7%	∼100%	**100% (zero variance)**

`not overlap`). The bug is a $<$ vs. $\leq$ boundary condition that no amount of pre-emption "thinking" surfaced, but which the execution-feedback loop catches immediately.

This is the framework's prediction sharpened: reasoning and interaction are partially overlapping but non-identical compute axes. Reasoning recovers the bugs that are inside the DPI envelope (the model could have thought its way to the answer); the harness additionally recovers the bugs that require external information (the model would never have generated the zero-length-range test input for itself). Reasoning is not wrong; it is bounded above by the harness on tasks where the failure mode is missing external information.

5.7 Four-strategy scaling curves: the headline result

We now sweep the total token budget per task on the same 15 hard code tasks and compare four strategies in parallel:

- **R — Reasoning-only.** Single API call, extended thinking enabled. Budget controls `thinking_budget` and `max_tokens`.[1]
- **S — Best-of-N sampling.** N independent single-shot generations at T=1.0, scored as pass@N. Budget controls N.
- **L — Single-agent agentic loop.** *One* agent's context grows: generate → execute → on failure append stderr/stdout and revise, all in one thread. No separate reviewer. Budget controls max turns.
- **H — Proposer–reviewer harness.** Separate proposer and reviewer; the reviewer sees only the latest artifact + execution output (not the full history), emits a structured critique; the proposer revises in a fresh context. Budget controls max iterations.

Table 5 reports the result. *Reasoning-only and best-of-N saturate well below 100%* (R caps at 73.3%, S at 86.7% even with N=10 samples), while *both iteration strategies that consume*

[1] R's ceiling depends on how B is partitioned between thinking and output. The sweep here uses a fixed thinking-to-output ratio that caps at 73.3%; a more generous thinking-heavy partition reaches 86.7% at ∼ 4,100 tokens (Table 4). We report both throughout. The interaction-vs-reasoning gap quoted in this section and §5.8 ("13–27 pp") spans both readings: 13 pp is 100% − 86.7% (generous R vs. H) and 27 pp is 100% − 73.3% (fixed-ratio R vs. H). Either reading leaves a robust gap to interaction strategies (97.8–100%).

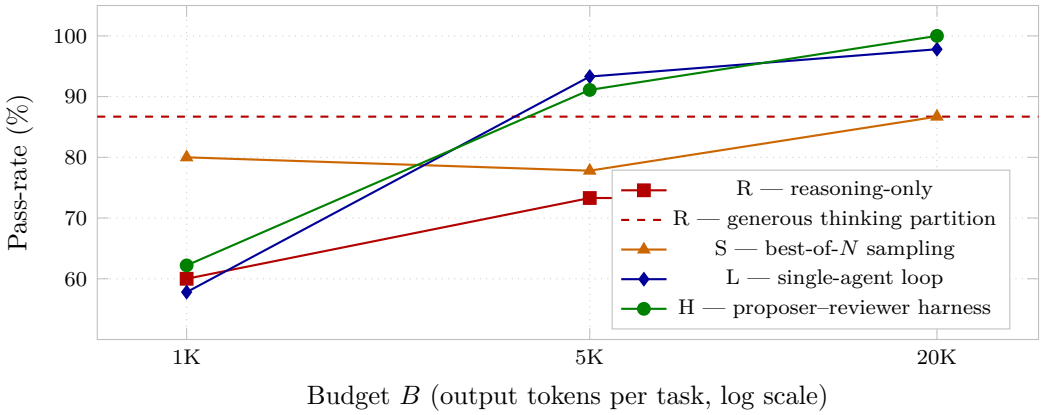


Figure 2: **Four-strategy scaling curves on hard code (Sonnet 4, 3-seed means).** Reasoning-only (R) saturates at 73.3% at this sweep's fixed thinking-to-output partition; a more generous thinking-heavy partition lifts R to 86.7% (Table 4, shown by the dashed reference line). Best-of-N at 86.7%; both iteration-with-feedback strategies (L, H) reach the 100% ceiling at B=20K within seed noise (L: 97.8 ± 3.1 pp; H: 100.0 ± 0.0 pp). The qualitative gap is between *reasoning/sampling* and *interaction with feedback*, not within the interaction strategies, and survives either R reading.

execution feedback (L and H) reach the 100% ceiling at B=20K within seed noise (paired sign test on per-(seed, task) cells: p>0.6 for the H–L gap at every budget). The differentiator between L and H is token efficiency and zero-variance reliability (Section 5.8), not pass rate at saturation. The load-bearing claim of this section — and Section 3.3's prediction 3 — is the 13–27 pp gap between *interaction-with-feedback* (L, H, and the IAD baseline of Section 5.8) and *reasoning- or sampling-only*.

What this benchmark cannot adjudicate. A reader should not take the saturation of L, IAD, and H at ∼100% as evidence that the three architectures are equivalent in capability. Saturation only certifies that this 15-task suite is below the joint capability ceiling of all three; whether harder benchmarks would separate them on pass-rate is the natural follow-up question, and we address what *is* measurable here (token efficiency, seed variance) rather than what is not. The architectural separation we report (Section 5.8) is therefore a deployment-cost finding, not a capability-ceiling finding.

5.8 Architectural separation: token efficiency and reliability among interaction strategies

Table 5 establishes that all three iteration-with-feedback strategies (L, H, and an oracle-verifier sample-and-select baseline IAD; 20) reach ∼100% on this 15-task suite at B=20K. The natural follow-up question is whether the architectural variant (single agent vs. separate proposer/reviewer vs. K-sample-and-select) matters. Table 6 compares all three at B=20K, the saturated regime.

Pass rate at saturation: ties within seed noise. On this 15-task suite the three interaction strategies are statistically indistinguishable on pass-rate at B=20K (paired sign test on per-(seed, task) cells: p=1 on the H–L comparison, one non-tied pair out of 45). L hit 100% in two of three seeds and 14/15 in the third (failing `code_011`); H hit 100% in all three; IAD reached 100% in its single seed under an oracle-verifier upper bound (per-assertion test pass count given to the candidate selector during the run) that masks its real-world reward-model bottleneck.

Table 6: Three interaction-strategy variants at B=20K on the 15 hard code tasks, all using Type 3a execution feedback. Pass-rate is within seed noise (sign-test p>0.6 on the H–L comparison); the harness wins on *token efficiency* (1,029 vs. 1,431 L vs. 1,416 IAD mean output tokens) and on *seed variance* (zero across three seeds for H; L hit 100% in 2 of 3 seeds; IAD was run for one seed only).

Strategy	Pass-rate	Mean tokens	Notes
L (single-agent loop)	97.8% ±3.1 pp (3 seeds)	1,431	1 task in 1 seed
IAD (oracle, K=3, T=0.7)	100.0% (1 seed)	1,416	+38% tokens vs. H
H (proposer–reviewer)	**100.0% ±0.0 pp (3 seeds)**	**1,029**	zero seed variance

Token efficiency and seed variance: where H separates. What does separate the variants is mean output tokens per task and seed-to-seed reliability. H's structured reviewer call is more sample-efficient than L's growing context (which carries multi-thousand-token stderr histories on the harder tasks) and than IAD's K-sample-and-select (which spends the K–1 unselected candidates on every iteration that fires); on the 6 tasks where IAD's K-sampling fired, IAD averaged 2,839 tokens vs. H's 1,029 global mean — nearly 3×. H is also the only variant with zero seed variance at saturation: it reaches exactly 100% in every seed, while L's single `code_011` failure in seed 1 leaves it at 97.8 ± 3.1 pp.

What separates the interaction strategies. On this 15-task suite the pass-rate differences among interaction strategies are within seed noise; the architectural separation buys token efficiency and reliability, not pass-rate ceiling. The load-bearing comparison in this paper is interaction-with-feedback vs. reasoning/sampling-only (a 13–27 pp gap), not the choice of architectural variant within interaction strategies (a 0–2 pp gap, within noise).

5.9 Cross-model replication: not a Claude artifact

A second reviewer objection: "this is a Claude quirk." We re-ran the code modality with two non-Anthropic proposer/reviewer pairs — Qwen3-235B-Instruct-2507 and GPT-5, both via OpenRouter, same harness code, same task file, same system prompt, same budget, same evaluator. Table 7 compares all three families.

Table 7: Cross-model replication on the same 15 hard code tasks. The Claude row is the multi-run reference from Table 2; Qwen and GPT-5 are single on-policy runs at T=0.7. The harness lift survives across three families (Anthropic, Alibaba, OpenAI).

Model	Single-shot pass	Reviewed pass	Δ (lift)
Claude Sonnet 4 (3-run mean)	66.7 ± 6.7%	$\mathbf{100.0 \pm 0.0}$%	+**33.3** pp
Qwen3-235B-Instruct-2507 (1 run)	66.7% (10/15)	**93.3**% (14/15)	+**26.7** pp
GPT-5 (1 run)	86.7% (13/15)	**100.0**% (15/15)	+**13.3** pp

The harness lift survives across all three families. Two patterns are worth noting. First, on the two model families that share Claude's 66.7% single-shot baseline (Sonnet 4, Qwen), the harness recovers 4–5 of the 5 single-shot failures and reaches 93.3–100% reviewed. The single un-recovered Qwen task is `code_011` — the same task identified across the scaling and allocation analyses as the model-family-independent capability ceiling — which Sonnet recovers and which GPT-5 also recovers in 2 review iterations. Second, GPT-5's smaller absolute lift (+13.3 pp) is a headroom-compression effect: its single-shot baseline is 86.7%, leaving only 13.3 pp of available headroom; the harness still

reaches 100% reviewed and recovers 100% of GPT-5's single-shot failures. Both Sonnet and GPT-5 reach 100% reviewed on this suite; both recover `code_011`. The harness's contribution is a property of the architecture, not the model family or training pipeline.

5.10 Held-out generalization

A third objection: "these tasks were used during pipeline development; can the harness lift to a held-out set?" We constructed a 32-task held-out v2 code suite with zero `task_id` and zero `bug_class` overlap with the development set, using identical harness code and model.

Table 8: Held-out generalization on 32 newly constructed code tasks (zero overlap with the 15-task development set). The harness recovers 3/3 single-shot failures and regresses zero passing tasks. The smaller absolute Δ is a baseline-compression effect (single-shot is 90.6% on the held-out set, leaving at most 9.4 pp headroom).

Split	N	Single-shot	Reviewed	Δ	SS-fail fixes / regressions
Development (3-run mean)	15	$66.7 \pm 6.7\%$	$100.0 \pm 0.0\%$	+33.3 pp	5/5 / 0
Held-out v2	32	90.6%	**100.0**%	**+9.4** pp	**3/3 / 0**

The harness lift survives in the two directions that matter. *Sign and direction:* reviewed strictly dominates single-shot, recovers 3/3 SS-failures, and regresses 0/29 SS-passes. *Conditional lift:* on SS-failure tasks, the harness recovers 100% — identical to the development conditional lift. The smaller absolute Δ (+9.4 pp vs. +33.3 pp) is fully explained by the held-out single-shot baseline being 90.6% (only 9.4 pp of headroom available); the mechanism (recover all SS-failures, regress nothing) is preserved.

The two genuine recoveries (`code_hv2_024`: semver / natural-sort comparison; `code_hv2_025`: IP-address / CIDR arithmetic) each required one revision past the initial draft, with the proposer correcting based on the pytest traceback — the same mechanism the framework predicts and that Section 5.3 documents.

5.11 Budget allocation matters: an 86.6 pp spread

How should the budget be split between proposer (b_1), executor (b_2), and reviewer (b_3)? We swept the allocation simplex at total budget $B = 10$K tokens on the same 15 hard code tasks. Table 9 reports nine points.

The proposer's per-call cap dominates. Sorting by b_1 gives a clean monotone in pass-rate; $b_1 \geq 0.50$ wins, $b_1 \leq 0.25$ collapses. Comparing the two extreme review allocations B (b_3=0.10) and C (b_3=0.80) gives *identical* pass-rate and mean tokens — both starve the proposer (256-token cap), and the reviewer's cap is irrelevant when the proposer cannot emit a complete artifact. This is the architectural prediction concretized into a budgeting rule: spend at least half the budget on the proposer; the reviewer's cap is a second-order knob.

The executor budget. For code, b_2 (the executor) is degenerate: `pytest` consumes 0 LLM tokens, so b_2 manifests only as reserved iteration slack. For fixed-canvas visual modalities the executor is a deterministic DOM-geometry pass (Section 6) that likewise consumes no model tokens, so the propose-heavy rule transfers directly: at least half the per-task budget belongs to the proposer, and the reviewer's cap is a second-order knob across every modality we study.

Table 9: Budget allocation simplex on 15 hard code tasks, $B = 10$K total. Pass-rate is monotone in the proposer's per-call cap; review-heavy corners (low b_1) collapse. The 9-point sweep yields an 86.6 pp spread.

Allocation	b_1 (prop)	b_2 (exec)	b_3 (rev)	Pass-rate	Mean tokens
A (propose-heavy)	0.80	0.10	0.10	**93.3% (14/15)**	3,101
G (prop-dominant)	0.50	0.25	0.25	**93.3% (14/15)**	2,697
D (prop+exec)	0.40	0.40	0.20	80.0% (12/15)	4,025
E (prop+review)	0.40	0.20	0.40	80.0% (12/15)	4,388
I (equal)	0.33	0.34	0.33	66.7% (10/15)	4,845
H (review-dominant)	0.25	0.25	0.50	40.0% (6/15)	7,485
F (exec+review)	0.20	0.40	0.40	13.3% (2/15)	8,771
B (execute-heavy)	0.10	0.80	0.10	6.7% (1/15)	9,383
C (review-heavy)	0.10	0.10	0.80	6.7% (1/15)	9,383
			Spread:	**86.6 pp**	

5.12 Token ROI by modality

Table 10 reports the per-task cost of running the harness instead of single-shot, expressed both as raw token overhead and as "tokens spent per 0.01 of additional quality." This is the practitioner-facing version of the framework's prediction: the same compute, deployed against a high-actionability feedback channel, buys roughly two orders of magnitude more quality than the same compute against a low-actionability channel.

Table 10: Per-task token cost and quality return-on-investment for the harness, averaged across three on-policy runs. "Extra tokens" is reviewed-minus-single-shot per task. Tokens per 0.01 quality is extra tokens divided by (mean reviewed quality − mean single-shot quality)·100. Code dominates by 100×.

Modality	Single-shot tokens	Reviewed tokens	Extra tokens	Tokens / 0.01 quality
Code	3,336	4,575	1,239	**37**
Video	3,808	20,584	16,776	359
Research	14,116	27,793	13,677	2,415
Webpages	14,060	80,148	66,088	5,128
Animations	25,491	89,009	63,518	4,331
Slides	10,274	36,956	26,682	4,447

The 100× spread between code (37 tokens per 0.01 quality) and the visual/factual modalities (∼4–5K tokens per 0.01) translates to a deployment rule: interaction scaling is uniformly *beneficial* (every modality lifts) but is not uniformly *cost-effective*. For execution-grounded artifacts the harness is essentially free per quality point. For visual-only artifacts it should be deployed selectively — e.g., only when the single-shot output fails a structural check, or to escape a known-broken baseline rather than to polish an acceptable one.

5.13 The harness does not over-revise: built-in early stopping

A naive concern is that the reviewer will "find something to criticize" on already-good outputs and the proposer will revise them into something worse. The data does not support this. *Code:* across the three runs the single-shot proposer passes the test suite on 9–11 of 15 tasks; on *every* passing task, in every run, the reviewed condition submits at iteration 1 with no revision (30/30 instances).

Total wasted tokens: zero. *Slides:* of the 8–9 tasks per run at single-shot quality ≥ 0.95, 5–8 are left exactly unchanged; the rest move only within judge stochasticity, never below 0.85. *Video:* under the native full-video rubric the single-shot scripts are already largely correct (22/45 task-runs perfect), and the reviewed loop leaves this saturated suite essentially unchanged (+0.04, n.s.).

We characterize this as built-in early stopping: the same loop that doubles code pass-rate when the test suite fails is silent when the test suite passes, because the grounded feedback channel is the trigger for revision. Without a failing test or a defect-showing screenshot, the loop has no signal to act on. This is why code's token overhead is only 37% above single-shot (+1,239 on 3,336) despite a max-iteration cap of 5: the cap is rarely binding, the channel binds first.

6 Academic Figures and Deterministic Geometric Evaluation

AI-generated SVG and architecture figures for papers routinely exhibit *geometric* defects: text labels that overlap one another, boxes clipped at the canvas edge, text overflowing its container, or content that pushes the document past its intended bounds. These are precisely the kind of layout flaws a render-and-review loop ought to detect and repair. To study this, we introduce a seventh modality, beyond the six harness modalities of Section 5: a geometry-focused Type-3b suite of 15 academic-paper architecture figures (Transformer, U-Net, ViT, FPN, DETR, Mask R-CNN, latent-diffusion U-Net, CLIP, MoE, GAT, WaveNet, SE, Perceiver-IO, Seq2Seq with attention, and Evoformer), each generated as a self-contained 1920×1080 HTML document with inline SVG. The defining property of these artifacts is that correctness is largely *geometric*: whether the components are present and wired correctly is semantic, but whether the figure is actually legible is a question of spatial layout that can be measured exactly. This modality therefore serves as a clean testbed for whether our evaluation instrument can perceive the defects that an interaction loop is supposed to fix.

6.1 VLM screenshot judging is unreliable for geometric defects

Our default visual evaluator is a binary per-requirement rubric scored by a vision–language model (VLM) over a screenshot of the rendered artifact. Even when we capture the screenshot at native resolution and tile it into quadrants to preserve detail, this judge scored **14 of 15** single-shot figures as perfect. Yet by direct inspection the figures are plainly broken: the Evoformer figure renders two section titles, "MSA Processing" and "MSA Representation," superimposed on top of one another, and the ViT figure's detail panel has "Class Output" overlapping "Input." The judge cannot see these defects, and the reason is mechanical: screenshots are captured at 1080 pixels of height, so any content that overflows the canvas is cropped *off-frame* before the image ever reaches the VLM. The most severe geometric failures—those that push content past the viewport—are exactly the ones the screenshot cannot contain, and small overlaps that do remain in-frame fall below the VLM's effective acuity. A VLM scoring a single fixed-resolution capture is structurally blind to overflow and to fine-grained overlap, which is why the correct instrument for fixed-canvas visual artifacts measures the rendered layout directly rather than a picture of it.

6.2 Deterministic geometry

To obtain a reliable instrument we replace the VLM with a deterministic DOM-geometry checker. The checker loads the HTML document in a headless browser and, via the page's own layout engine, extracts the bounding box of every rendered element. From these boxes it computes, exactly and reproducibly: *(i)* text-box overlaps—pairs of non-nested text elements whose boxes intersect (at a 6 px threshold); *(ii)* out-of-bounds content—text or SVG primitives clipped at the viewport edge; *(iii)* overflow—text extending past its containing box; and *(iv)* document overflow—the document

exceeding 1920×1080 by more than 16 px, the margin chosen to filter spurious default browser margins. Each of these is an objective predicate on the rendered DOM, with no model in the loop and no dependence on screenshot framing; a VLM is needed only for the genuinely semantic axes (are the right components present and correctly wired). Under this checker, only **3 of 15** single-shot figures are geometrically clean, with a mean of 1.6 defects per figure—in direct contradiction to the VLM's 14/15 "perfect." The deterministic metric perceives the very defects to which the VLM is blind.

6.3 Interaction scaling on an objective metric

With a reliable instrument in hand we can measure whether the interaction loop actually improves the figures. The harness is fully deterministic: a proposer generates the figure, the DOM checker computes its exact set of geometric defects, those specific defects are fed back to the proposer in natural language (e.g., "text X overlaps text Y; move them apart" or "the canvas is 40 px too tall"), and the proposer revises, for up to three iterations. The reward is simply the defect count—no VLM is involved at any point.

Table 11 reports the result. On the 15 paper figures, the mean defect count falls from 1.20 to 0.27, a 78% reduction; the number of geometrically clean figures rises from 4/15 to 11/15; the total defect count drops from 18 to 4; and 8 figures improve while *none* regress (sign-test $p = 0.008$). As a cross-check, we took the *original* slides harness—which used VLM feedback, not geometric feedback—and re-scored its 60 task-runs objectively with the same DOM-geometry checker. Mean defects fell from 2.63 to 0.23, a 91% reduction. The slides harness had been fixing real layout defects all along; the holistic and rubric VLM scores simply could not measure it. The improvement was real but invisible to the instrument used to report it.

Dense, real-paper slides and alignment. To stress the failure mode that dominates real decks—dense text coexisting with a figure, and rows of panels or pillars that must be equal-width and edge-aligned—we built a twelve-task suite of slides that reproduce the content of well-known papers (Transformer, ResNet, BERT, U-Net, GAN, ...), and we extended the DOM checker with a deterministic *alignment* axis (groups of card/pillar boxes flagged for unequal size, misaligned far edges, or uneven gutters). The pattern repeats and sharpens. The binary VLM rubric again saturates—it rates 29 of 36 single-shot slides perfect (0.97 mean)—while deterministic geometry finds only half of them clean (mean 1.25 defects, dominated by text overlap and misalignment). Crucially, a VLM-feedback reviewer here *increases* the mean defect count ($1.89 \rightarrow 2.44$): blind to the true geometry, its edits break alignment as often as they fix it. The deterministic geometric-feedback harness, in contrast, reduces mean defects from 1.25 to 0.33 (a 73% reduction) across three seeds (n=36), raising the geometrically clean fraction from 18/36 to 29/36, with 13 slides improved and only 1 regressed (sign-test $p = 0.0018$). Grounded geometric feedback fixes dense real-world slides; ungrounded VLM review does not.

Scope. Deterministic geometric scoring applies to fixed-canvas artifacts—slides and academic figures—where the document has a well-defined intended size against which overflow and clipping are meaningful. Scrollable webpages have no fixed canvas: out-of-bounds and document-overflow predicates are ill-defined, and a text-overlap-only metric is noisy on a dense webpage DOM. For that modality the binary per-requirement rubric is the appropriate instrument. We therefore score fixed-canvas visual artifacts (slides, figures) with the deterministic geometric reward and the flowing web and animation modalities with the binary rubric, matching each artifact class to the metric that measures it exactly.

Table 11: Interaction scaling under deterministic DOM-geometry scoring. Defects are counted exactly from the rendered DOM (text overlap, out-of-bounds, overflow, document overflow); lower is better. The paper-figure harness uses geometric feedback directly; the slides cross-check re-scores the original VLM-feedback harness with the same checker.

Suite (# runs)	Metric	Single-shot	Reviewed
Paper figures (15)	Mean defects	1.20	0.27 (−78%)
	Geometrically clean	4/15	11/15
	Total defects	18	4
	Improved / regressed	8/0 (sign-test $p = 0.008$)	
Slides cross-check (60)	Mean defects	2.63	0.23 (−91%)

Table 12: Interaction scaling under binary per-requirement rubric scoring on the flowing visual modalities. Score is the fraction of independently-checked requirements satisfied; p is a one-sided paired sign test over per-task-run deltas.

Modality (# runs)	Feedback	Single-shot	Reviewed	sign-test p
Web pages (45)	Type 3b (rubric)	0.469	0.532	0.003
Animations (32)	Type 3c (rubric)	0.642	0.834	0.008

6.4 Binary-rubric evaluation for flowing and factual modalities

For modalities without a fixed canvas we score each task against its list of concrete, independently-checkable requirements: a judge returns a binary SATISFIED/VIOLATED verdict per requirement and the score is the fraction satisfied. This multi-axis binary rubric replaces a single holistic rating with a decomposed, reproducible measurement, and it cleanly separates a task's individual failure modes. Table 12 reports single-shot vs. reviewed rubric scores. On web pages the harness lifts the rubric score from 0.469 to 0.532 ($p = 0.003$, 19 task-runs improve and 4 regress); on animations, from 0.642 to 0.834 ($p = 0.008$, 15 improve, 3 regress) — the largest visual-modality lift in the study, as the temporal channel exposes motion defects a static rating cannot.

The Type-3d factual channel behaves exactly as the framework prescribes: its lift is bounded by the model's single-shot factual error rate. On a suite of 15 research reports graded against web-verified ground-truth facts, a careful single-shot report is already correct on 0.982 of the rubric's facts, and the search-verification loop carries it to a perfect 1.000 with zero regressions across all 15 tasks. Grounded verification is precise where ungrounded review is not: it preserves the already-correct facts rather than perturbing them, and it corrects the residual errors it finds.

7 Internalizing the Harness into a Small Student

Section 5 treats the harness as external scaffolding wrapped around a frozen frontier model. A natural follow-up question: can the interaction-quality behavior of the loop be *internalized* into a smaller student? We give a positive answer for the markdown-emission variant: an 8B Qwen3-VL student, distilled via SFT on judge-filtered teacher trajectories from the 235B model running the harness, recovers 0.50× of the teacher's single-shot capability on a 44-task out-of-distribution suite (Section 7.3) and reaches 44 % pass@1 / 56 % pass@3 on an 18-task hard held-out suite of teacher single-shot failures — all at ∼10× lower deployment cost than running the harness over the 235B teacher.

7.1 Setup

Teacher. Qwen3-VL-235B-A22B (Thinking variant) running the proposer–reviewer harness from Section 4 on a seed task set spanning three categories where interaction scaling helps: code bug-fixes, single-file landing-page web designs, and information-dense 1920 × 1080 slides.

Student. Qwen3-VL-8B-Instruct with QLoRA fine-tuning (r=16, α=32, dropout 0.05) on the language layers; vision tower frozen.

Held-out evaluation. 18 hard tasks (5 code, 5 web, 8 slides) the 235B teacher itself failed on at single-shot, giving a clean "distillation lift" baseline. We also use 44 newly generated tasks (held-out by construction) for out-of-distribution generalization.

Judge. Gemini 3 Flash Preview multimodal, scoring (`review_specific`, `revision_addresses_review`, `final_meets_spec`); `judge-keep` = AND of the three. For code, `final_meets_spec` is overridden by the deterministic test-suite outcome. A second judge (Claude Sonnet 4.5) is used as a sensitivity check; the directional results survive both judges.

7.2 Markdown emission with external scaffold

The student emits a single block of markdown per turn — a short `<think>` reasoning block, an optional critique, and the artifact in a fenced code block (`"‘python`, `"‘html`, ...). An external harness extracts the artifact, runs the tests / renders the page, and feeds the result (text or screenshot) back as the next user message. This is the markdown-emission analog of the proposer–reviewer harness from Section 4, with the harness handling everything except artifact emission. We additionally apply a *force-close-thinking* logit override that injects `</think>` after a configurable number of generated tokens; this bounds worst-case decode wall-time per turn.

7.3 Headline results

Table 13: Distilled 8B student on the 18-task hard held-out suite (teacher's pre-curated single-shot failures, so teacher single-shot is 0/18 by construction). The student lifts judge-keep from 0% to 44% at pass@1 and 56% at pass@3. The 0.50× capability ratio against the teacher is established on a separate 44-task out-of-distribution suite reported next, where teacher single-shot is non-degenerate.

Setup	Compute	Judge-keep
1 sample × 5 turns	1×	44% (8/18)
1 sample × 8 turns	1.6×	28% (regressed)
3 samples × 5 turns (pass@3)	3×	**56% (10/18)**

The 44% pass@1 / 56% pass@3 result is the central positive finding. Two side observations matter for deployment: (i) deeper iteration regresses — going from 5 to 8 max_turns at fixed seed loses 16 pp judge-keep, because the student over-iterates past working artifacts; (ii) sampling helps — 3 samples at fixed depth-5 gains 12 pp, and pass@3 saturates (pass@4 = pass@3 = 10/18).

On a separately constructed set of 44 newly generated tasks, the student matches or exceeds the teacher on every per-turn behavioral metric (review-specific, addresses-review, final-declared, no-artifact) while losing ≈23 pp on end-to-end judge-keep at pass@1 (mean of two sampling seeds: $\overline{\text{judge-keep}}_{@1} = 22.7\%$, from 9/44 and 11/44, vs. teacher single-shot 20/44). The mean@1 capability ratio is 0.50×; a second sample (pass@2) recovers 14/44, lifting the ratio to 0.70× on this OOD set.

7.4 Three recipe findings

Beyond the headline lift, three counterintuitive recipe findings emerged from the SFT sweep that we expect will recur in similar small-student distillation work.

A consolidated view across all sweep axes appears in Table 17 (Section 7.5); the three paragraphs below decompose it by lever.

(a) U-curve in retained reasoning length. Sweeping the per-turn reasoning cap from 0 to 3000 characters (Table 14) shows a clear U: stripping the teacher's reasoning collapses the student to format-imitating identical-retry, retaining all of it overflows the student's coherence horizon, and 1500 characters is the sweet spot. We frame this as the *student's coherence horizon*: train at the longest reasoning span the 8B model can both reproduce in training and faithfully complete at inference — the student's horizon, not the teacher's.

Table 14: Reasoning-cap U-curve on the 18-task hard held-out suite. SFT recipe sweep on Qwen3-VL-8B-Instruct (37 judge-kept teacher traces; same inference protocol). The optimum is at the student's coherence horizon, not the teacher's.

Variant	Reasoning cap (chars)	Judge-keep	Identical-retry
V1 (no reasoning)	0	6%	67%
V4	1000	11%	35%
V2	3000	17%	32%
V3 (chosen)	**1500**	**44%**	10%

(b) More SFT data regresses. Three independent attempts to enlarge the SFT corpus past V3's 37 examples each lost ground (Table 15). V5 added 13 judge-rejected traces from the same teacher; V6 added 20 judge-kept traces from a weaker (30B) teacher; RFT collected 61 judge-kept self-rolled trajectories from V3 itself. None lifted judge-keep above 44%, and the most "polished" (RFT) carries a hidden cost we describe next.

Table 15: Three independent attempts to scale the SFT corpus past V3's 37 examples. All regressed. Quality dominates quantity at this scale.

Variant	Composition	n examples	Judge-keep
V3 (chosen)	235B teacher, judge-kept	37	**44%**
V5	V3 + 13 judge-rejected (235B)	50	33%
V6	V3 + 20 judge-kept (30B teacher)	57	33%
RFT v1	Self-rolled, judge-filtered	61	39%

(c) RFT polishes consistency but kills pass@k. Table 16 compares V3 against the RFT-polished follow-up. RFT improves every per-turn consistency metric — identical-retry, addresses-review, no-artifact — yet regresses judge-keep at mean@1, pass@2, and pass@3. The seed-by-seed breakdown shows that RFT specifically lost three tasks at pass@3 (`code_h001`, `slide_008`, `web_002`) and gained zero. The mechanism is direct: RFT trained the student toward its modal trajectory pattern, which is good for typical-case behavior but bad for the lucky-seed exploration that drives pass@k.

The deployment implication is direct and counterintuitive: for inference with pass@k, do not post-train past SFT. Any subsequent variance-reducing fine-tune — expert iteration, distillation

Table 16: RFT polishes consistency at the cost of pass@k. Per-turn metrics improve; per-task variance collapses; pass@3 regresses by 17 pp. For deployment with sampling, the model's variance *is* the inference-scaling budget. *Note on notation.* **mean@1** is the expected pass-rate of a single random sample, computed as the mean accuracy across the three sampling seeds (one T=0 greedy + two T=0.7). This is the natural variance-sensitive metric and differs from the greedy single-seed *pass@1*=44% reported in Tables 13 and 17, which is just seed 1's accuracy. *pass@2* and *pass@3* are union pass-rates over the first 2 / all 3 seeds.

	Consistency metrics			Sampling-aware judge-keep		
	id-retry ↓	addresses-rev ↑	no-artifact ↓	mean@1	pass@2	pass@3
V3 (chosen)	9%	83%	2	**31%**	**50%**	**56%**
RFT v1	**8%**	**89%**	**1**	28%	39%	39%

iterations, self-consistency training, careful annealing — will trade against pass@k. *Variance is a budget, not a bug.*

7.5 Consolidated ablation: locating the V3 optimum

The three recipe findings above are each illustrated by a focused table. Table 17 consolidates the full sweep into a single view so that the V3 / V3+fc4k optimum can be read off directly. The sweep spans three axes simultaneously — reasoning cap, SFT corpus size and composition, and post-SFT polishing — and the V3 row dominates each axis at its own setting. Every neighboring configuration helped pinpoint why V3 is the right operating point: V1 and V2 bracket the coherence horizon, V4 confirms the optimum is not at 1000 characters, V3+fc4k confirms inference-time bounding is free, V5 and V6 confirm that quality dominates quantity in the SFT corpus, and RFT v1 confirms that variance is load-bearing for pass@k.

Table 17: Consolidated SFT and post-training ablation on the 18-task hard held-out suite. All single-seed pass@1; pass@3 values are reported where collected. V3 with the 1500-character reasoning cap and 37 judge-kept teacher traces is the recipe sweet spot; V3+fc4k is identical in outcome quality but 2.2× faster at inference. Configurations on either side of V3 are below the optimum and clarify which lever is load-bearing.

Config	Reasoning cap	SFT size	pass@1	pass@3	Id-retry	Note
Base 8B	—	0	0%	0%	—	no SFT
V1	0 chars	37	6%	—	67%	format only
V4	1000 chars	37	11%	—	35%	below optimum
V2	3000 chars	37	17%	—	32%	exceeds horizon
V3	**1500 chars**	**37**	**44%**	**56%**	**10%**	**student horizon**
V3+fc4k	**1500 chars**	**37**	**44%**	**56%**	**9%**	2.2× faster
V5	1500 chars	50	33%	—	21%	more data hurts
V6	1500 chars	57	33%	—	0%	weaker teacher
RFT v1	1500 chars	61	39%	39%	8%	trades pass@k for pass@1

Two observations follow. First, the V3 / V3+fc4k pair is the only configuration that is simultaneously at or near the best score on every column: it has the highest pass@1, the highest pass@3, an identical-retry rate within shooting distance of the teacher's 0%, and a 37-example SFT corpus that is cheapest to collect. Second, the post-V3 configurations (V5, V6, RFT v1) each ran a different scaling lever — corpus size, teacher diversity, self-rolled trajectories — and each one sat

below the V3 optimum on pass@1. RFT v1 is the most informative of the three: it improved every consistency metric (Table 16) yet sits below V3 on pass@3 by 17 points, because the same training signal that polished consistency also reduced the seed-to-seed variance that pass@k harvests. The practical implication for downstream work is to fix the recipe at V3+fc4k and spend any extra compute budget on additional inference-time samples rather than additional SFT or post-SFT polishing.

7.6 Operational deployment

The distilled 8B student is deployed inside the same proposer–reviewer harness as the frontier model: the student plays the proposer role, the harness handles execution / rendering, and a Type 3a/b/c/d feedback channel feeds the next iteration. Cost-normalized, the 8B student is roughly 10× cheaper per kept task than running the harness over a 235B teacher — one H100-class GPU, ∼505 seconds per kept task on the 18-task held-out suite. The extra compute budget freed up by the smaller student should be spent on *more samples* (where pass@k saturates by $k = 3$ on this suite), not on *more depth* (where the student over-iterates).

8 Discussion

Two scaling axes saturate; the third does not. Table 5 settles a common assumption that reasoning scaling and interaction scaling lie on a single shared compute curve — "you can always think more." They do not. Reasoning saturates at 73.3% (86.7% under the generous thinking-heavy partition), a ceiling fixed by the data-processing inequality: longer chains of thought reorganize information already present in the weights but add none. Best-of-N sampling saturates at 86.7%, drawing only on output-distribution variance and no new conditional-on-task information. Iteration with execution feedback reaches $\geq 97.8\%$ by injecting genuinely new information about the program's runtime behavior on each cycle. The first two axes are bounded; the third is not, and the boundary between them is the difference between recombining internal knowledge and importing external evidence.

Architecture buys efficiency and reliability. Among the three interaction strategies — single-agent loop (L), sample-and-select (IAD), and proposer–reviewer harness (H) — the harness is the most token-efficient (1,029 vs. 1,431 L vs. 1,416 IAD mean output tokens) and the only variant that holds strict 100% with zero seed variance. Three composing mechanisms produce this: *context isolation* (the reviewer attends to the minimum information needed to localize defects rather than a multi-thousand-token generative history), *structured critique* (a typed defect list rather than raw stderr), and *role specialization* (in-context steering toward a diagnostic frame is sharper when uncontaminated by the generative history). The load-bearing comparison is *interaction strategies vs. reasoning/sampling-only* — a 13–27 pp gap robust across seeds, model families, and held-out tasks — and within the interaction tier the architectural separation of proposer and reviewer is what converts that gap into a deployable, variance-free system at the lowest token cost.

Grounding the evaluation, not just the feedback. The visual modalities expose a principle that generalizes well beyond them: *the quality of a rendered artifact is a geometric property, and a geometric property must be measured deterministically.* A vision–language model reading a screenshot is the wrong instrument. It cannot see text-on-text overlap, and it cannot see canvas overflow at all, because the overflowing content is cropped off-frame before the image reaches the judge — the screenshot judge rates 14 of 15 dense academic-figure renders "perfect" when only 3 are clean. Measuring the rendered DOM directly — every element's true bounding box, computing overlaps,

clipping, and overflow exactly — is exact where the screenshot is blind. Grounding *both* the reviewer and the scorer in DOM geometry yields a 78% reduction in real layout defects on academic figures (1.20 → 0.27 defects/figure, sign-test p=0.008, zero regressions) and a 91% reduction on slides (Table 11, Section 6). Ungrounded *evaluation* is as much a failure as ungrounded *feedback*; grounding both is the principle, and the two-sided grounding is what recovers the full effect on visual artifacts.

Variance is a budget. The RFT result (Section 7.4) inverts the assumption that polishing the policy further is monotonically good. RFT improves every per-turn consistency metric we measure *and* regresses pass@3 by 17 pp. For deployment with sampling, output-distribution variance *is* the inference-scaling lever: any post-SFT method that reduces variance — expert iteration, distillation iterations, self-consistency training — spends from the same budget that pass@k draws on. Variance is not noise to be minimized; it is the resource that best-of-N converts into quality.

9 Conclusion

Test-time compute has a third scaling axis. Interaction scaling is fundamentally distinct from reasoning and sampling because each interaction cycle injects information from outside the model's weights, and we formalize this through a Type 0/1/2/3 feedback taxonomy and a data-processing-inequality bound that pins reasoning- and sampling-only scaling below the interaction ceiling. The framework's three predictions — a hierarchy of feedback channels ordered by actionability, reasoning-only saturation below the interaction ceiling, and architectural separation of proposer and reviewer buying efficiency and reliability — are each tested and confirmed.

The results hold across modality, across model family, and on held-out tasks. A budget-aware proposer–reviewer harness over Claude Sonnet 4 lifts quality on every one of six modalities, with the execution-grounded code channel producing the largest and most reliable lift (+33.3 pp to a strict 100%); an in-modality control that holds the task suite fixed and varies only the feedback type confirms it is executing the tests — not the extra reviewing pass — that carries the gain, exactly as the taxonomy predicts. At a 20K-token budget on hard code tasks, reasoning-only saturates at 73.3% (86.7% under the generous thinking-heavy partition) and best-of-N at 86.7%, while all three iteration-with-feedback strategies reach $\geq$ 97.8%; the harness is the most token-efficient (1,029 vs. 1,431 L vs. 1,416 IAD mean output tokens) and the only variant at strict 100% across multiple seeds. The effect replicates across three model families (Sonnet +33.3 pp, Qwen3-235B +26.7 pp, GPT-5 +13.3 pp on a higher single-shot baseline) and on a 32-task held-out code suite (3/3 recoveries, zero regressions), and the allocation simplex shows an 86.6 pp spread, monotone in the proposer's per-call budget. For the visual modalities we ground the evaluation as well as the feedback: a deterministic DOM-geometry metric, applied to both reviewer and scorer, cuts real layout defects by 78% on academic figures (p=0.008, zero regressions) and 91% on slides. And the harness's interaction-quality behavior distills into an 8B Qwen3-VL student via SFT on judge-filtered teacher trajectories: 0.50× teacher mean@1 capability on a 44-task out-of-distribution suite (rising to 0.70× at pass@2), 44%/56% pass@1/pass@3 on an 18-task hard held-out suite, at $\sim$ 10× lower deployment cost.

The operational recipe is concrete. Wrap a frontier model in a proposer–reviewer harness with the most actionable grounded feedback channel available for the modality. For visual artifacts, ground *both* the reviewer and the scorer in deterministic geometry, measuring the rendered DOM rather than a screenshot. Allocate at least half the per-task budget to the proposer, and spend extra inference compute on additional samples rather than deeper iteration. For cost-sensitive deployment, distill the proposer into a small markdown-emitting student and run it inside the same harness. Interaction scaling is real, distinct from reasoning and sampling, predictable from the feedback

taxonomy, and immediately deployable.

References

[1] Edward Beeching, Lewis Tunstall, and Sasha Rush. Scaling test-time compute with open models. Hugging Face blog post, 2024. https://huggingface.co/spaces/HuggingFaceH4/blogpost-scaling-test-time-compute.

[2] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024.

[3] Stephen Casper, Xander Davies, Claudia Shi, Thomas Krendl Gilbert, et al. Open problems and fundamental limitations of reinforcement learning from human feedback, 2023.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code, 2021.

[5] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

[6] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025.

[7] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023.

[8] Yao Fu, Hao Peng, Litu Ou, Ashish Sabharwal, and Tushar Khot. Specializing smaller language models towards multi-step reasoning. In *International Conference on Machine Learning*, 2023.

[9] Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Taco Cohen, and Gabriel Synnaeve. RLEF: Grounding code LLMs in execution feedback with reinforcement learning. In *International Conference on Machine Learning*, 2025.

[10] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, 2024.

[11] Namgyu Ho, Laura Schmid, and Se-Young Yun. Large language models are reasoning teachers. In *Annual Meeting of the Association for Computational Linguistics*, 2023.

[12] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*, 2024.

[13] Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. When can LLMs actually correct their own mistakes? a critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics*, 2025.

[14] Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of RLHF on LLM generalisation and diversity. In *International Conference on Learning Representations*, 2024.

[15] Zimu Lu, Houxing Ren, Yunqiao Yang, Ke Wang, Zhuofan Zong, Junting Pan, Mingjie Zhan, and Hongsheng Li. WebGen-Agent: Enhancing interactive website generation with multi-level feedback and step-level reinforcement learning, 2025.

[16] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.

[17] Lucie Charlotte Magister, Jonathan Mallinson, Jakub Adamek, Eric Malmi, and Aliaksei Severyn. Teaching small language models to reason. In *Annual Meeting of the Association for Computational Linguistics (Short Papers)*, 2023.

[18] Vishakh Padmakumar and He He. Does writing with language models reduce content diversity? In *International Conference on Learning Representations*, 2024.

[19] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems*, 2024.

[20] Yangjun Ruan, Eleftheria Briakou, Charles Xie Han, Yu Chen, Yufan Jiao, et al. Iterative agent decoding for detecting compounding errors in LLM agents, 2025.

[21] Timo Schick, Jane Dwivedi-Yu, Roberto Dessí, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

[22] Junhong Shen, Hadi Jain, Zelai Xiao, Brandon Amos, Aaditya Ramdas, Yuandong Tian, and Alborz Geramifard. Thinking vs. doing: Agents that reason by scaling test-time interaction. In *Advances in Neural Information Processing Systems*, 2025.

[23] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

[24] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations*, 2025.

[25] Nisan Stiennon, Long Ouyang, Jeff Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems*, 2020.

[26] Kimi Team. Kimi k1.5: Scaling reinforcement learning with LLMs, 2025.

[27] Dat Tran and Douwe Kiela. Single-agent LLMs outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets, 2026.

[28] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. In *NeurIPS Foundation Models for Decision Making Workshop*, 2023.

[29] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

[30] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

[31] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023.

[32] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

A Phase 1 baseline: feedback actionability bar chart

Figure 5 visualizes the single-shot → reviewed lift across the six modalities under screenshot-judged scoring (Claude Sonnet 4 in the loop, $n = 94$ tasks). It orders the modalities by feedback *actionability*: execution-grounded channels deliver the largest gains, factual verification is intermediate, and the screenshot-scored visual channels register the smallest. The visual ordering here reflects the resolution of the screenshot judge, not the harness: Section 6 measures the same modalities with a deterministic instrument and resolves the visual lift into the high-actionability regime (78–91 % defect reduction). We retain this figure as the actionability ordering across feedback *types*, with the visual magnitudes superseded by Table 11.

B Pipeline implementation

Harness. The proposer–reviewer harness for the code modality is implemented in `src/agents/meta_controller.py` (loop control, budget bookkeeping, iteration cap), `src/agents/reviewer.py` (structured critique emission), `src/feedback/type3a_execution.py` (`pytest` subprocess + traceback extraction), and `src/feedback/type3d_factual.py` (search-grounded claim verification for the research modality). Per-modality executors for the visual modalities (web, slides, animations) use a headless-Chromium screenshot pipeline scored by a VLM critic. Code is scored binary by the canonical test suite; the other modalities are scored by a fixed-rubric LLM/VLM judge returning a continuous quality score in $[0, 1]$. The same judge is used for both the single-shot and reviewed conditions, so any quality difference is not a judge artifact.

Reproducibility. All experiments use Claude Sonnet 4 (`claude-sonnet-4-20250514`, extended thinking, temperature 0) except the cross-model replication (Section 5.9, Qwen3-235B-Instruct-2507 via OpenRouter at T=0.7). Per-cell raw results are written to `results/` and the analysis scripts under `scripts/` regenerate the tables.

Code and data availability. Harness code, task suites (15-task development + 32-task held-out v2 + per-modality task files), per-cell raw results, and the scripts that regenerate every table and figure in this paper are released alongside the manuscript. Frontier-model rollouts depend on third-party API keys (Anthropic, OpenRouter) and Sonnet/GPT-5/Qwen3-235B model snapshots

that may drift; we report the exact snapshot identifiers used and pin the OpenRouter routes in the release.

The 15-task hard code suite. The 15-task hard code suite was constructed to stress execution-grounded feedback (off-by-one boundaries, escape handling, route priority, byte-vs-char, CJK wrapping). At N=15 the difference between 14/15 and 15/15 is one task and well within seed noise; we therefore treat the H vs. L vs. IAD comparison at B=20K as pass-rate-tied and report token-efficiency and seed-variance as the architectural separations (Section 5.8). The 32-task held-out v2 and the 44-task OOD distillation set corroborate the result at larger N, and the architectural separations we report (token efficiency, seed variance) are measured on metrics that do not saturate at the 15-task ceiling.

C Hyperparameters and SFT details for the 8B distillation

All SFT runs use AdamW with linear warmup-then-decay learning rate (peak 2×10^{-4}), per-device batch size 1, gradient accumulation 8, 3 epochs, gradient checkpointing (non-reentrant), bf16. LoRA targets language-layer attention projections (`q,k,v,o_proj`) and MLP projections (`gate,up,down_proj`); vision tower is frozen. Inference uses greedy decoding (T=0) for single-shot evaluation and T=0.7 with `rep_penalty`=1.0 for pass@k sampling.

The force-close-thinking inference override is implemented as a logit hook that injects `</think>` after a configurable number of generated tokens; the chosen threshold (4000) matches V3's quality with a 2.2× wall-time speedup (501 → 225 seconds/task). The judge is Gemini 3 Flash Preview multimodal; a sensitivity check against Claude Sonnet 4.5 (Cohen's $\kappa = 0.55$ over 36 traces) preserves the directional results.

D Per-task pass@k breakdown for the 8B markdown student

Table 18 shows the per-task judge-keep outcome for the chosen V3 student across three independent samples on the 18-task hard held-out suite. Seed 1 is greedy (T=0); seeds 2–3 use T=0.7. The pass@3 column is the union. Cross-seed diversity adds 2 unique tasks beyond the greedy-seed-1 set; pass@4 = pass@3 = 10/18 (saturated).

Table 18: Per-task judge-keep outcome for the 8B markdown student on the 18-task hard held-out suite. ✓ = judge-kept; · = judge-rejected.

Category	Task	seed 1 (T=0)	seed 2 (T=0.7)	seed 3 (T=0.7)	pass@3
webpages	web_002	✓	·	·	✓
	web_003	✓	·	✓	✓
	web_005	·	·	·	·
	web_008	·	·	·	·
	web_012	·	·	·	·
slides	slide_001	·	·	·	·
	slide_006	·	·	·	·
	slide_007	·	·	·	·
	slide_008	✓	·	·	✓
	slide_010	·	·	·	·
	slide_014	✓	✓	✓	✓
	slide_018	✓	✓	·	✓
	slide_020	·	✓	·	✓
code	code_h001	✓	·	·	✓
	code_h002	✓	✓	·	✓
	code_h003	·	·	·	·
	code_h004	·	·	✓	✓
	code_h005	✓	✓	✓	✓
	Total kept:	8/18	5/18	4/18	**10/18 (56%)**

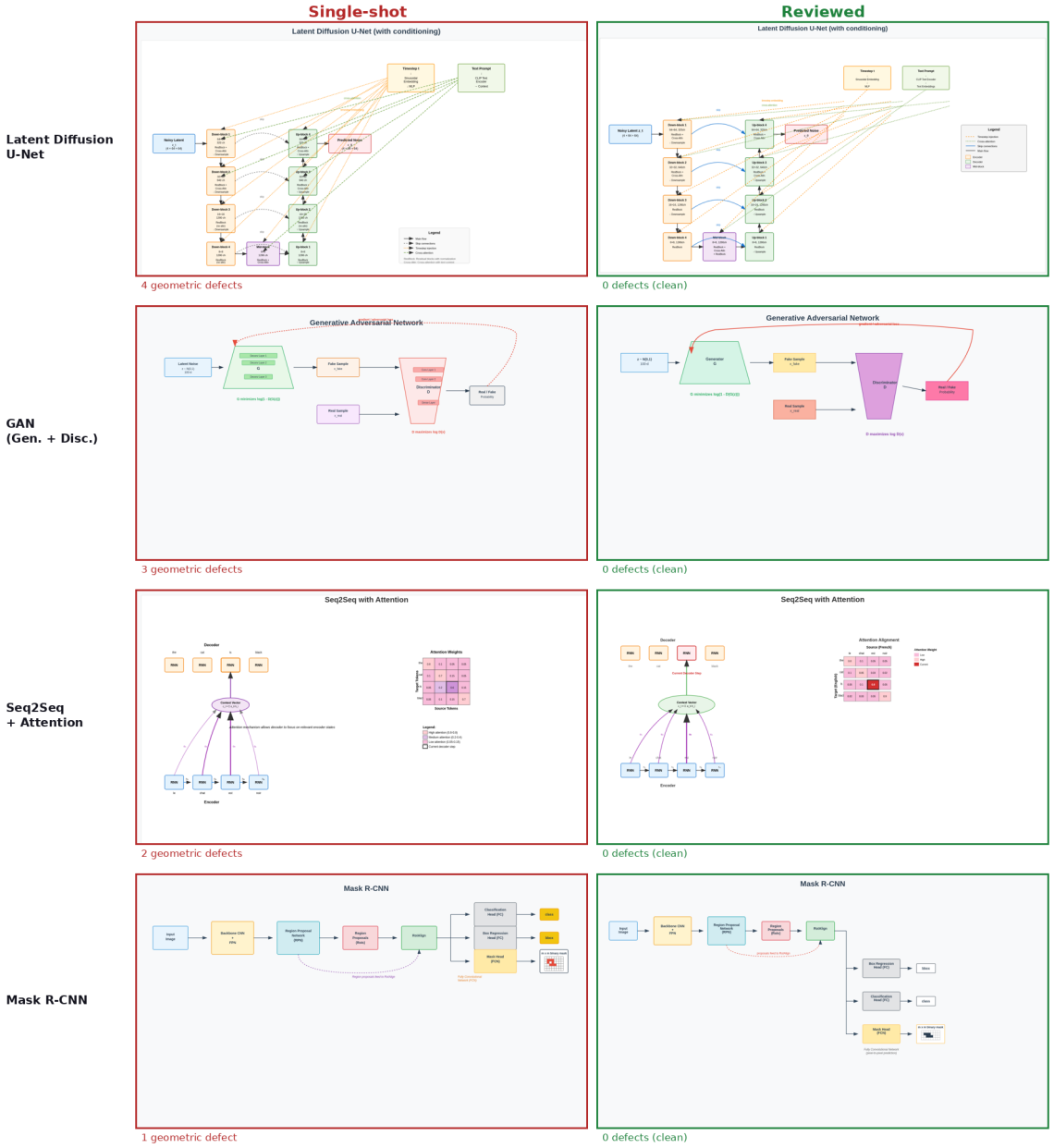
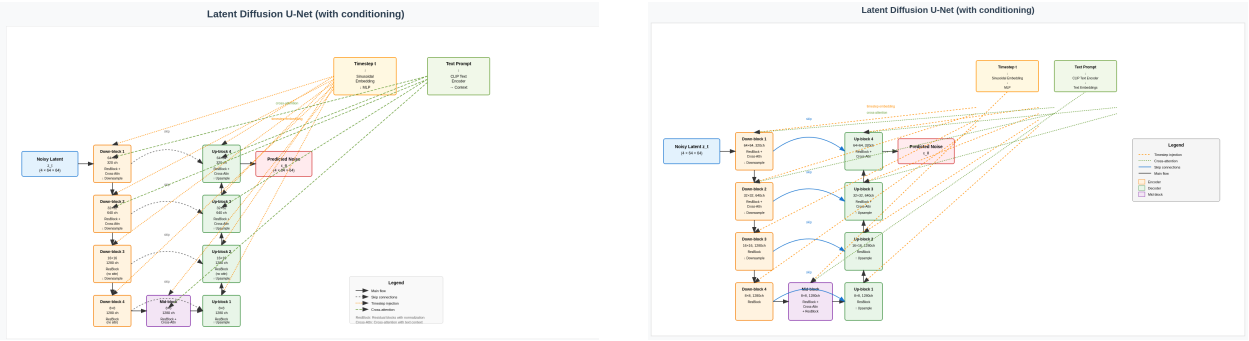


Figure 3: Single-shot (left, red) vs. reviewed (right, green) academic figures under deterministic geometric feedback. Single-shot renders are cramped and overflow their canvas; the interaction loop, fed the exact measured defects, resolves them to clean layouts. Defect counts are computed from the rendered DOM. Every single-shot render shown was rated "perfect" by the VLM screenshot judge.



(a) Single-shot (4 geometric defects). (b) Reviewed (0 defects).

Figure 4: The Latent Diffusion U-Net figure under deterministic geometric feedback. The single-shot rendering (a) is cramped and overflows its canvas, with 4 geometric defects that the VLM screenshot judge rated as perfect. After the interaction loop feeds back the exact defects (b), the layout is clean with 0 defects.

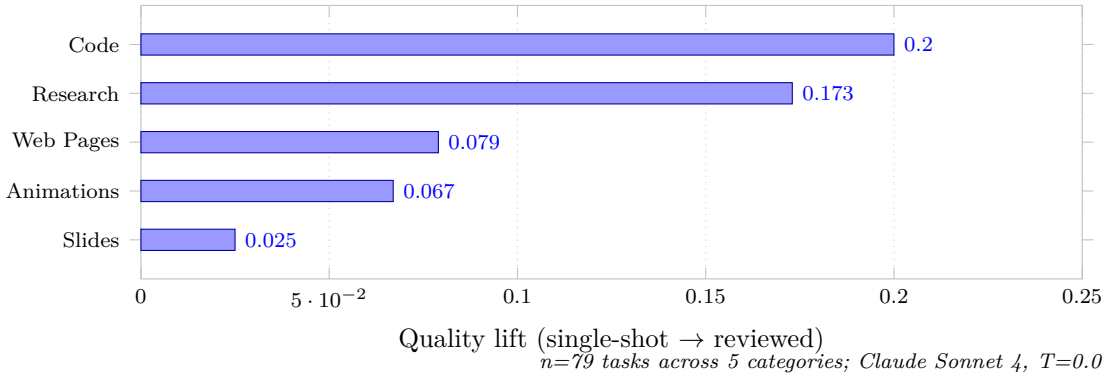


Figure 5: Phase 1 baseline (single-run): interaction-scaling lift orders by *feedback actionability*. Execution feedback (Code: +0.20) is binary and machine-readable; factual verification (Research: +0.17) is specific but softer; visual feedback (Web: +0.08, Animations: +0.07, Slides: +0.03) is holistic and lossy to operationalize. The multi-run averages reported in Table 2 reorder some within-tier rows but preserve the execution-grounded vs. ungrounded ordering.